

UNIVERSIDAD NACIONAL DE LA MATANZA

Programación Concurrente – 3658

TRABAJO PRÁCTICO INTEGRADOR

#	Integrante
1	FERNANDES, LEONEL JULIAN
2	MANGANO ROSENFELD, ALAN EITAN
3	MAUDET, ALEJANDRO NICOLÁS
4	CASCASI, AXEL JOEL

1. Repositorio GitHub	3
2. Descripción del Proyecto	3
2.1 Finalidad del Software	3
2.2 Lenguaje y Frameworks	3
2.3 Requisitos del Sistema	5
2.4 Restricciones y Dependencias	5
2.5 Arquitectura General	5
3. Manual de Usuario	7
3.1 Instalación	7
3.2 Configuración Inicial	7
3.3 Uso de la Aplicación	8
3.4 Video demostrativo	10
4. Conclusiones	11
4.1 Dificultades Encontradas	11
4.2 Aprendizajes	11
4.3 Reflexiones Finales	12



1. Repositorio GitHub

El código fuente completo del proyecto se encuentra disponible en el siguiente repositorio:

[Repositorio Grupo M5](#)

[Tag Final v1.1.0](#)

2. Descripción del Proyecto

A continuación se presenta el README.md con la descripción completa de la solución implementada.

2.1 Finalidad del Software

Sheriffsss es un videojuego de vista superior desarrollado en **Java** cuyo objetivo principal es ofrecer un entorno interactivo de entrenamiento para practicar movimiento, puntería y uso de armas dentro de una arena controlada. La aplicación permite al jugador desplazarse por un mapa, apuntar con el mouse, disparar a dianas, seleccionar armas y recibir retroalimentación inmediata mediante HUD, sonidos, efectos visuales, contador de aciertos, fallos, precisión y puntaje final.

El problema que resuelve dentro del alcance del trabajo práctico es la construcción de una simulación gráfica en tiempo real que integre entrada de usuario, renderizado, audio, persistencia de configuración y coordinación de tareas concurrentes sin bloquear la experiencia de juego. Para ello, el sistema separa el ciclo principal de actualización, el renderizado **Swing/AWT**, el manejo de audio sincronizado y los componentes propios del modo entrenamiento, manteniendo una ejecución fluida orientada a 60 FPS.

En síntesis, el software funciona como una base jugable y extensible para validar mecánicas de combate, entrenamiento y configuración, aplicando conceptos de programación concurrente en un contexto práctico, visual e interactivo.

2.2 Lenguaje y Frameworks

- **Lenguaje:** Java 17 o superior.
- **Frameworks / Librerías:** **Java** estándar, principalmente **Swing/AWT** para interfaz gráfica y renderizado, **javax.sound.sampled** para audio, **ImageIO** para carga de imágenes y librerías estándar de Java para archivos, colecciones y configuración. No utiliza frameworks externos.
- **Herramientas de build:** Se usan scripts propios con javac para compilar. Además, incluye *build-portable-windows-exe.bat*, que genera un paquete portable para



Windows con un .exe, el .jar de la aplicación y un runtime Java embebido mediante jlink, por lo que **la PC destino no necesita tener JDK ni JRE instalado**.

Como opción alternativa, se puede ejecutar el proyecto con *compile-and.run.bat*, se requiere tener instalado **JDK 17 o superior**, ya que los scripts compilan el código fuente con javac antes de iniciar el juego.



2.3 Requisitos del Sistema

- **Sistema Operativo:** Windows, Linux y macOS si se compila con Java/JDK; el paquete portable con `.exe` aplica únicamente a Windows.
- **Hardware mínimo:**
 - **CPU:** procesador de doble núcleo de 64 bits.
 - **RAM:** 2 GB mínimo, recomendado 4 GB.
 - **GPU:** gráficos integrados compatibles con aplicaciones Java AWT/Swing. No requiere GPU dedicada.
 - **Almacenamiento:**
 - Aproximadamente 100 MB para el proyecto en código fuente.
 - Aproximadamente 300 MB para el paquete portable con `.exe`.
 - **Recomendado:** 500 MB libres.
- **JDK / Runtime:**
 - *Para compilar o ejecutar desde código fuente:* JDK 17 o superior.
 - *Para ejecutar el `.jar`:* Java Runtime / JRE 17 o superior, o JDK 17+.
 - *Para el distributable portable de Windows con `.exe`:* no requiere instalar JDK ni JRE, porque incluye un runtime Java embebido generado con `jlink`.
- **Navegador (si aplica):** No aplica.

2.4 Restricciones y Dependencias

La aplicación es de ejecución local y no requiere conexión a Internet, servicios externos ni base de datos. No utiliza frameworks externos, librerías de terceros, servidores, APIs remotas ni motores gráficos adicionales. Todo el funcionamiento del juego se resuelve con recursos incluidos en el proyecto: código Java, sprites, sonidos y archivos de configuración.

Como dependencia principal necesita un entorno Java 17 o superior para compilar o ejecutar desde código fuente. En el caso del paquete portable para Windows, esta dependencia queda resuelta dentro del propio distributable mediante un runtime embebido, por lo que no es necesario instalar Java en la máquina destino.

2.5 Arquitectura General

La solución está organizada como una aplicación Java de escritorio basada en Swing/AWT. El punto de entrada es Main, que crea la ventana principal (JFrame) y carga el componente central Game, encargado de coordinar el ciclo de juego, la entrada del usuario, los estados de pantalla y la comunicación entre los distintos sistemas.

La arquitectura se divide por responsabilidades en paquetes:

SheriffsssPackage

— context	→ configuración global, input, audio, assets, ciclo día/noche y debug
— session	→ estado mutable de la partida: mapa, jugador, equipo, nivel activo y runtime
— level	→ lógica del modo entrenamiento, construcción de arena y control de sesión
— system	→ sistemas de gameplay: movimiento, armas, proyectiles, enemigos, música
— render	→ renderizado del mundo, menú, HUD, efectos visuales y modelos de vista



└─ ui → controladores de interfaz, cursores, menú de armas y opciones de pantalla

El flujo general separa la inicialización, la simulación y la presentación visual. Game mantiene el loop principal en un hilo propio orientado a 60 FPS; en cada iteración procesa input, actualiza el estado de la sesión, mueve al jugador, actualiza enemigos/proyectiles, controla música y finalmente solicita el repintado de la pantalla. El renderizado queda delegado a GameRenderer y MenuRenderer, que leen el estado mediante la interfaz GameView, evitando que la capa visual modifique directamente la lógica del juego.

El estado del juego se maneja principalmente a través de GameSession, que contiene el mapa activo, el jugador, el zoom de cámara, el modo de entrenamiento y el nivel actual. Las dependencias estables, como AssetManager, AudioManager, GameInput, EnemySystem y ProjectileSystem, se agrupan en GameContext, lo que permite mantener separada la configuración general del estado cambiante de cada sesión.

A nivel funcional, el modo entrenamiento se construye desde TrainingSessionBuilder y TrainingMode. Este módulo genera una arena cerrada, posiciona al jugador, crea dianas, controla aciertos/fallos, temporizador, precisión, reinicio de arena y datos del HUD. Los sistemas de armas y proyectiles se encargan de calcular disparos, cooldowns, precisión, trayectoria, daño e impacto sobre objetivos.

En términos de concurrencia, la aplicación diferencia el hilo de interfaz de Swing del hilo principal del juego. Además, el manejo de audio usa métodos sincronizados para evitar conflictos al reproducir, detener o liberar clips desde distintos momentos del ciclo de ejecución. Esta separación permite que la simulación, la interfaz y los recursos audiovisuales funcionen de forma coordinada sin bloquear la interacción del usuario.



3. Manual de Usuario

Esta sección describe paso a paso cómo instalar, configurar y utilizar la solución.

3.1 Instalación

1. Clonar el repositorio oficial del proyecto:

```
> git clone https://github.com/UNLAM-PROG-C/2026-PROGC-Q1-M5.git
```

2. Ingresar al directorio del proyecto y posicionarse en la versión final utilizada para la entrega:

```
> cd 2026-PROGC-Q1-M5  
> git checkout develop
```

3. Acceder a la carpeta del juego:

```
> cd TP-Integrador/Sheriffsss
```

4. Si se ejecuta desde código fuente, verificar que esté instalado JDK 17 o superior:

```
> java -version  
> javac -version
```

5. Compilar y ejecutar la aplicación según el sistema operativo:

Windows PowerShell:
> .\compile-and-run.ps1

Windows CMD:
> compile-and-run.bat

Linux / macOS:
> ./compile-and-run.sh

6. Como alternativa, si ya se cuenta con el .jar generado, se puede ejecutar directamente:

```
> java -jar dist-jar/Sheriffsss.jar
```

7. En Windows también puede utilizarse el paquete portable generado con build-portable-windows-exe.bat, que incluye un ejecutable y un runtime Java embebido. En ese caso, no es necesario instalar JDK ni JRE en la máquina destino.

3.2 Configuración Inicial



La aplicación no requiere configuración previa compleja, variables de entorno obligatorias, conexión a Internet ni base de datos. De forma opcional, puede configurarse **JAVA_HOME**, aunque no es obligatorio si el **JDK** ya está agregado al **PATH**.

La estructura del proyecto debe mantenerse sin alterar, especialmente las carpetas:

- **resources/** → sprites, cursores, imágenes y sonidos del juego
- **saves/** → archivos de configuración persistente
- **src/** → código fuente Java

La carpeta **resources/** debe estar disponible para que el juego pueda cargar correctamente los assets visuales y de audio. La carpeta **saves/** se utiliza para guardar preferencias locales, como resolución, pantalla completa, volumen de música, volumen de efectos y cantidad de dianas del modo entrenamiento.

Los archivos de configuración principales son:

- **saves/game.cfg** → configuración general de pantalla y audio
- **saves/training.cfg** → configuración del modo entrenamiento

Estos archivos pueden existir previamente o generarse/actualizarse automáticamente durante la ejecución. Por lo tanto, la aplicación necesita permisos de lectura sobre los recursos y permisos de escritura dentro de **saves/**.

3.3 Uso de la Aplicación

3.3.1 Menú Principal

Al iniciar la aplicación se muestra el menú principal de Sheriffsss. Desde allí el usuario puede comenzar el modo entrenamiento, acceder a la configuración general o salir del juego.





3.3.2 Modo Entrenamiento

El modo entrenamiento es la funcionalidad principal del sistema. El jugador aparece en una arena cerrada donde puede practicar movimiento, puntería y disparo contra dianas. La sesión mide aciertos, fallos, precisión y puntaje final durante un tiempo limitado.

Durante la partida, el usuario puede aumentar o reducir la cantidad de dianas, reiniciar la arena, saltar el tutorial inicial o volver al menú principal.

3.3.3 Movimiento y Cámara

El jugador se mueve con las teclas W, A, S y D. La cámara sigue al personaje y puede acercarse o alejarse mediante +, - o Ctrl + rueda del mouse. Esto permite adaptar la visibilidad del mapa según la preferencia del usuario.

3.3.4 Armas y Disparo

El jugador apunta con el mouse y dispara con click izquierdo. El sistema calcula la dirección del disparo, aplica precisión del arma, cooldown, velocidad del proyectil, daño e impacto sobre las dianas.

El inventario de armas puede abrirse con TAB, y las armas disponibles se seleccionan con las teclas numéricas o con la rueda del mouse.

3.3.5 HUD y Resultados

Durante el entrenamiento se muestra un HUD con información relevante de la sesión: aciertos, fallos, precisión, temporizador, mensajes de guía y resultado final. Al terminar el tiempo, la aplicación muestra el puntaje obtenido y permite reiniciar el entrenamiento o volver al menú principal.

3.3.6 Configuración

Con ESC se accede al menú de configuración. Desde allí se puede modificar volumen de

música, volumen de efectos, resolución de ventana y pantalla completa. Estas preferencias se guardan automáticamente en saves/game.cfg.

En modo entrenamiento también puede habilitarse un panel de debug desde la configuración, utilizado para visualizar información técnica como hitboxes, trayectorias, conos de precisión y otras ayudas de prueba.

3.3.7 Controles Principales

Acción	Control
Mover jugador	W, A, S, D
Apuntar	Mouse
Disparar	Click izquierdo
Acción secundaria	Click derecho
Abrir/cerrar armas	TAB
Cambiar arma	Rueda del mouse
Pausa/configuración	ESC
Pantalla completa	F11 o Alt + Enter
Zoom de cámara	+, -, Ctrl + rueda
Aumentar dianas	PgUp
Disminuir dianas	PgDown
Reiniciar entrenamiento	R
Volver al menú	B

3.4 Video demostrativo

[Video Sheriffsss](#)



4. Conclusiones

En esta sección el grupo describe las dificultades encontradas durante el desarrollo del trabajo práctico, los aprendizajes obtenidos y reflexiones finales.

4.1 Dificultades Encontradas

Uno de los principales fue coordinar correctamente el ciclo de juego, la entrada del usuario, el renderizado en Swing/AWT y la reproducción de audio sin bloquear la interfaz ni generar comportamientos inconsistentes.

También resultó complejo organizar el crecimiento del código. Al incorporar mapa, jugador, armas, proyectiles, dianas, HUD, configuración y modo entrenamiento, fue necesario separar responsabilidades en paquetes y sistemas específicos para evitar que toda la lógica quedara concentrada en una única clase.

Además, se presentaron desafíos relacionados con la carga y distribución de recursos. El juego depende de sprites, sonidos, cursores y archivos de configuración, por lo que fue necesario mantener una estructura de carpetas estable y contemplar distintos modos de ejecución: desde código fuente, desde .jar y mediante paquete portable para Windows.

En cuanto a lo no técnico, una dificultad fue coordinar el trabajo grupal sobre distintas ramas, revisiones y versiones del proyecto, manteniendo una integración coherente entre funcionalidades desarrolladas por diferentes integrantes.

4.2 Aprendizajes

Durante el desarrollo se pusieron en práctica conceptos de programación concurrente aplicados a una aplicación gráfica en tiempo real. El caso principal fue la separación entre el hilo de interfaz de Swing/AWT y el hilo propio del juego, donde se ejecuta el ciclo principal de actualización. Esto permitió comprender la importancia de no bloquear la interfaz y de mantener una simulación fluida mientras se procesan entradas, renderizado y lógica de juego.

También se trabajó con sincronización en el manejo de audio. El uso de métodos `synchronized` en el administrador de sonido permitió controlar el acceso concurrente a recursos compartidos como clips, música de fondo, efectos de sonido y liberación de recursos al cerrar la aplicación.

Otro aprendizaje importante fue el uso de variables y operaciones seguras para coordinar el cierre del juego y la vida útil de hilos. Por ejemplo, el loop principal utiliza banderas de control para finalizar correctamente, evitando que queden procesos activos o recursos abiertos después de salir.

Además, el proyecto permitió entender que la concurrencia no solo consiste en crear hilos, sino en definir responsabilidades claras entre ellos. El estado principal del juego debe actualizarse de forma ordenada, mientras que tareas auxiliares como audio, eventos de interfaz o cierre de la aplicación deben coordinarse sin introducir condiciones de carrera.



En general, aprendimos a aplicar concurrencia en un contexto práctico: mantener una aplicación interactiva, responder al usuario en tiempo real, administrar recursos compartidos y diseñar una estructura que permita extender el juego sin volver inestable su ejecución.

4.3 Reflexiones Finales

Como grupo, consideramos que el trabajo práctico fue una experiencia valiosa porque permitió aplicar conceptos de programación concurrente en un proyecto más cercano a un caso real que a un ejercicio aislado. El desarrollo de un videojuego obligó a pensar en actualización constante, respuesta inmediata al usuario, administración de recursos compartidos y separación entre lógica, renderizado, audio e interfaz.

El proyecto también evidenció la importancia de organizar el código a medida que crece. A partir de la incorporación de nuevas mecánicas, fue necesario separar responsabilidades en paquetes y sistemas, lo que mejoró la mantenibilidad y dejó una base más clara para futuras extensiones.

Como posibles mejoras futuras, se podría continuar refinando la arquitectura, reducir clases demasiado grandes, agregar más modos de juego, nuevos enemigos, más armas, pruebas automatizadas y una configuración más flexible de niveles. También sería útil mejorar la validación del código respecto a estándares de estilo, reducción de números mágicos y límites de tamaño de métodos.

En conclusión, Sheriffsss cumple como una base jugable y funcional para demostrar integración entre interfaz gráfica, simulación en tiempo real y coordinación concurrente. Más allá del resultado final, el mayor aporte del trabajo fue entender cómo diseñar una aplicación interactiva donde varias partes del sistema deben ejecutarse de forma coordinada, estable y mantenible.

